



Unidade 2 — Frameworks



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Explicar** as motivações técnicas para o uso de Frameworks;
- **Comparar** o desenvolvimento com e sem Framework;
- **Analisar** o impacto de Frameworks na produtividade e qualidade;
- **Avaliar** cenários em que o uso de Framework é recomendado ou não.

Taxonomia de Bloom: Explicar, Comparar, Analisar, Avaliar.



Motivação 1: Eliminar Repetição

Sem framework — código repetido em cada projeto:

```
// Criar servidor (todo projeto)
const server = http.createServer(handler);
// Parse de JSON (todo projeto)
let body = '';
req.on('data', chunk => body += chunk);
req.on('end', () => { data = JSON.parse(body); });
// Servir arquivos estáticos (todo projeto)
fs.readFile('./public/index.html', (err, data) => { ... });
```

Com framework — código focado no que é único:

```
app.use(express.json());
app.use(express.static('public'));
app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});
```



Motivação 2: Padronização

Sem Framework	Com Framework
Cada dev organiza de um jeito	Convenções definidas pelo framework
Nomes inconsistentes	Padrões de nomenclatura
Estruturas diferentes	Estrutura de pastas padronizada
Dificulta colaboração	Facilita colaboração

Convenção sobre Configuração (Convention over Configuration): O framework define padrões razoáveis por padrão, reduzindo a necessidade de configuração manual.



Motivação 3: Comunidade e Ecossistema

- **Documentação** oficial e da comunidade;
- **Tutoriais** e cursos disponíveis;
- **Plugins** e extensões prontas;
- **Stack Overflow** com milhares de respostas;
- **GitHub** com issues e discussões;
- **Conferências** e meetups.

Dado: O Express.js possui mais de **60.000 estrelas** no GitHub e é utilizado por **milhões** de projetos.



Motivação 4: Segurança

Vulnerabilidade	Sem Framework	Com Framework
XSS	Implementar manualmente	Proteção built-in
CSRF	Implementar manualmente	Middleware pronto
SQL Injection	Implementar manualmente	ORM parametrizado
Helmet	Configurar manualmente	Middleware pronto

Frameworks populares são auditados por milhares de desenvolvedores. Vulnerabilidades são identificadas e corrigidas rapidamente.



Motivação 5: Escalabilidade

Projeto pequeno (sem framework):

- server.js (200 linhas)
- Funciona bem

Projeto médio (sem framework):

- server.js (2.000 linhas)
- Difícil de manter
- Precisa de organização

Projeto grande (sem framework):

- server.js (20.000 linhas)
- Impossível de manter
- Precisa de framework

Frameworks são projetados para escalar — de pequenos a grandes projetos.



Quando NÃO Usar um Framework?

Cenário	Recomendação
Projeto muito simples (1-2 rotas)	Pode ser overkill
Aprendizado de fundamentos	Prefira fazer manualmente primeiro
Requisitos muito específicos	Framework pode ser restritivo
Equipe sem experiência com o framework	Considere a curva de aprendizagem
Protótipo rápido (hackathon)	Pode usar abordagem mais simples

Princípio: Não use um framework apenas porque é popular. Use porque resolve o **seu problema**.



Síntese da Aula

Conceitos principais:

1. Frameworks eliminam **repetição** e fornecem infraestrutura pronta;
2. A **padronização** facilita colaboração e manutenção;
3. A **comunidade** oferece documentação, plugins e suporte;
4. Frameworks contribuem para **segurança** e **escalabilidade**;
5. Nem sempre um framework é a melhor escolha — avalie o **contexto**.

Pergunta reflexiva:

Se você pudesse escolher apenas UMA razão para usar um framework, qual seria? Por quê?

Próxima aula: Estrutura de um Framework — Vamos desmontar um framework para entender suas partes.

